山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验 11 搜索树		
实验学时：2	实验日期： 2023.11.29	

实验目的：

创建带索引的二叉搜索树类。存储结构使用链表，提供操作:插入、删除、按名次删除、查找、按名次查找、升序输出所有元素。

软件开发工具：

DEV C++

1. 实验内容

创建带索引的二叉搜索树类。存储结构使用链表，提供操作:插入、删除、按名次删除、查找、按名次查找、升序输出所有元素。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

所需要的数据结构与算法：带索引的二叉树，递归

整体思路描述：

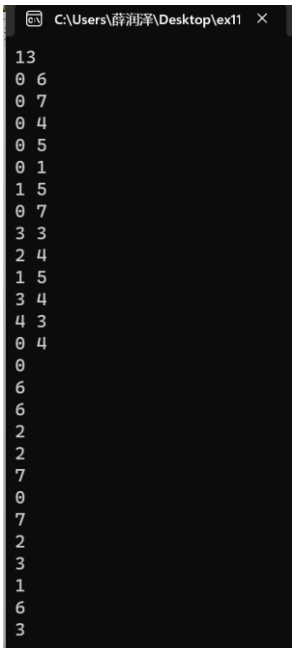
（1）插入操作：int push(Node*&cnt, int v, int ans = 0);函数先进行递归查找，如果 cnt 等于空指针，说明函数遍历到了树的叶子节点处，此时进行插入操作。如果 cnt->value == v，则说明树中已经存在 v 值，进行返回。如果 cnt->value < v 则遍历右子树，如果 cnt->value > v 则遍历左子树。同时在遍历左子树的过程中，如果最终成功插入了一个节点，则对左子树路径上的点进行更新操作。

（2）查找操作：int search(Node*&cnt, int v, int ans = 0);函数递归进行查找，如果 cnt 等于空指针，则说明没有找到相应元素，返回 0。如果 cnt->value 比要查找的元素的 value 值要小，则对其右子树进行遍历。否则对其左子树进行遍历。

（3）删除操作：int deleteElement(Node*&cnt, int v, int ans = 0);首先递归找到要删除的点。此时可以分为 4 种情况，要被删除点为叶子节点，要被删除点有左孩子无右孩子，要被删除点有右孩子无左孩子，要被删除点有两个孩子。上述四种情况中，前三种情况选择被删除点的父节点和被删除点有的唯一的那个孩子（叶子节点时无孩子，此时选择与 nullptr 相连接）连接，并删除被删除点。对于第四种情况，先找到要被删除节点右子树的最小节点，将该最小节点替换到被删除节点处，leftptr, rightptr, LeftSize 更新为被删除节点值。然后将被删除节点删除。要注意的是，由于函数参数中传入的是 Node*&cnt，也就是一个引用，所以我们在更新该节点与父节点的连接状况时，可以直接使用 cnt = ? 这样的表达来直接将父节点与该节点的子节点相连接。

（4）根据名次查找和删除：根据名次查找和删除与查找操作和删除操作相比，差异在查找上。充分利用在 push 操作和两个删除操作中一直更新维护的 LeftSize 值，通过 LeftSize 记录的每个节点左子树节点个数来计算该节点的相对名次。例如，在遍历左子树时，左子树的每一个点的名次由 LeftSize + 1 来计算出。当遍历右子树时，操作 Rank，使 Rank 减去遍历右子树前根节点的名次获得一个新 Rank，再将新 Rank 用于子树的名次比较中，子树名次计算由 LeftSize + 1 来计算出。

3. 测试结果（测试输入，测试输出）



4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

最后一个样本点要求优化输出来加快程序运行速度。可以选择使用代码std::ios::sync_with_stdio(false);std::cin.tie(nullptr);来解除 cin 与缓冲区的绑定，从而加速代码运行。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include<iostream>

struct Node{
    Node(int v){
        leftptr = nullptr;
        rightptr = nullptr;
        value = v;
        LeftSize = 0;
    }
    int value;
    int LeftSize = 0;
    Node* leftptr = nullptr;
    Node* rightptr = nullptr;
};
class   SearchTree{
public:
    SearchTree(){
        root = nullptr;
    }
    int push(Node*& cnt, int v,int ans = 0 ){//cnt 是一个引用
        if(cnt == nullptr){cnt = new Node(v); return ans;}//如果 cnt == nullptr，则进行插入
        else if(cnt->value == v) return -1;//原来树中已经存在 v 值
```

```
        else if(cnt->value < v) push(cnt->rightptr, v ,ans ^ cnt->value);
        else {//插入到左子树，RankNode++
        int finial = push(cnt->leftptr, v, ans ^ cnt->value);
        if(finial != -1) cnt->LeftSize++;//如果插入成功，更新 LeftSize 值
        return finial;}
    }
    void Push(int v){
        int ans = push(root, v);
        if(ans == -1)std::cout<<"0"<<'\n';//ans = -1 ，树中存在元素，输出 0
        else std::cout<<ans<<'\n';
        return;
    }
    int search(Node*&cnt, int v, int ans = 0){
        if(cnt == nullptr) return 0;//没找到，返回 0
        else if(cnt->value == v) return ans ^ v;//找到了，返回异或值
        else if(cnt->value < v) return search(cnt->rightptr, v, ans ^ cnt->value);
        else return search(cnt->leftptr, v, ans ^ cnt->value);
    }
    void Search(int v){
        int ans = search(root, v);
        std::cout<<ans<<'\n';
    }
    Node*& findMin(Node*& cnt){//开始时 cnt 是待删除节点的右子树根节点，保证初始不为空
        if(cnt ->leftptr == nullptr) return cnt;
        cnt->LeftSize--;
        findMin(cnt->leftptr);
    }
    int deleteElement(Node*&cnt, int v, int ans = 0){
        if(cnt == nullptr)return -1;
        else if(cnt->value < v) deleteElement(cnt->rightptr, v, ans ^ cnt->value);
        else if(cnt->value > v) {
            int finial = deleteElement(cnt->leftptr, v, ans ^ cnt->value);
            if(finial != -1) cnt->LeftSize--;//如果成功删除某点，则 LeftSize--
            return finial;
        }
        else if(cnt->value == v){//找到了要删除点
            ans = ans ^ cnt->value;
            if(cnt->leftptr == nullptr && cnt->rightptr == nullptr){//删除点为叶子节点
                delete cnt;
                cnt = nullptr;
            }
            else if(cnt->leftptr != nullptr && cnt->rightptr == nullptr){//有一个孩子
                Node* temp = cnt;
                cnt = cnt->leftptr;
                delete temp;
```

```
            }
            else if(cnt->leftptr == nullptr && cnt->rightptr != nullptr){
                Node* temp = cnt;
                cnt = cnt->rightptr;
                delete temp;
            }
            else if(cnt->leftptr != nullptr && cnt->rightptr != nullptr){//有两个孩子
                Node*& Mintemp1 = findMin(cnt->rightptr);//Mintemp 是最小节点的指针引
用
                Node* Mintemp = Mintemp1;
                Mintemp1 = Mintemp1->rightptr;//将原来父节点的右指针指向被替换点的右
结点
                Node* temp = cnt;//记录 cnt 指针，用于释放原 cnt
                Mintemp->leftptr = cnt->leftptr;
                Mintemp->rightptr = cnt->rightptr;
                Mintemp->LeftSize = cnt->LeftSize;
                cnt = Mintemp;
                delete temp;
            }
            return ans;
        }
    }
    void DeleteElement(int v){
        int ans = deleteElement(root, v);
        if(ans == -1) std::cout<<"0"<<'\n';
        else std::cout<<ans<<'\n';
    }
    int RankSearch(Node*cnt, int rank, int ans = 0){
        if(cnt == nullptr) return 0;//没找到
        if(cnt->LeftSize + 1 > rank) RankSearch(cnt->leftptr, rank, ans^cnt->value);
        else if(cnt->LeftSize + 1 < rank) RankSearch(cnt->rightptr, rank - cnt->LeftSize -
1,ans^cnt->value);
        else return ans^ cnt->value;//找到
    }
    void rankSearch(int rank){
        int temp = RankSearch(root, rank);
        std::cout<<temp<<'\n';
//        std::cout<<RankSearch(root, rank)->value<<'\n';
    }

    int RankDelete(Node*& cnt, int rank, int ans = 0){
        if(cnt == nullptr)return -1;//没找到,删除失败
        if(cnt->LeftSize + 1 > rank){//当前节点序号比要找的序号大
            int tempans = RankDelete(cnt->leftptr, rank, ans^cnt->value);//向当前节点的左侧
寻找
            if(tempans != -1) cnt->LeftSize--;//如果删除成功，则更新 LeftSize 值
```

```
            return tempans;
        }
        else if(cnt->LeftSize + 1 < rank) RankDelete(cnt->rightptr, rank - cnt->LeftSize - 1, ans^cnt->value);//当前节点比要找的序号小，向右寻找
        else{//找到，对 deleteElement 代码删除操作复用
            ans = ans ^ cnt->value;
            if(cnt->leftptr == nullptr && cnt->rightptr == nullptr){//删除点为叶子节点
                delete cnt;
                cnt = nullptr;
            }
            else if(cnt->leftptr != nullptr && cnt->rightptr == nullptr){//有一个孩子
                Node* temp = cnt;
                cnt = cnt->leftptr;
                delete temp;
            }
            else if(cnt->leftptr == nullptr && cnt->rightptr != nullptr){
                Node* temp = cnt;
                cnt = cnt->rightptr;
                delete temp;
            }
            else if(cnt->leftptr != nullptr && cnt->rightptr != nullptr){//有两个孩子
                Node*& Mintemp1 = findMin(cnt->rightptr);//Mintemp 是最小节点的指针引用
                Node* Mintemp = Mintemp1;
                Mintemp1 = Mintemp1->rightptr;//引用置为空，原来父节点的 leftptr 或者 rightptr 则为空
                Node* temp = cnt;
                Mintemp->leftptr = cnt->leftptr;
                Mintemp->rightptr = cnt->rightptr;
                Mintemp->LeftSize = cnt->LeftSize;
                cnt = Mintemp;
                delete temp;
            }
            return ans;
        }
    }
    void rankDelete(int rank){
        int ans = RankDelete(root, rank);
        if(ans == -1)std::cout<<"0"<<'\n';
        else std::cout<<ans<<'\n';
    }
    void preorder(Node* cnt){//测试用输出
        if(cnt == nullptr) return ;
        std::cout<<cnt->value<<' ';
        preorder(cnt->leftptr);
        preorder(cnt->rightptr);
```

```
    }
    void preLeftSize(Node* cnt){
        if(cnt == nullptr)return;
        std::cout<<cnt->LeftSize<<' ';
        preLeftSize(cnt->leftptr);
        preLeftSize(cnt->rightptr);
    }
    void Preorder(){
        preorder(root);
        std::cout<<'\n';
        preLeftSize(root);
    }
private:
    Node *root;
};
int main(){
    std::ios::sync_with_stdio(false);//解除 std::cin 的缓冲区绑定，加快 std::cin 的速度
    std::cin.tie(nullptr);
    SearchTree tree;
    int m;
    std::cin>>m;
    int a, b;
    for(int i = 1; i<= m; i++){
        std::cin>>a>>b;
        if(a == 0){
            tree.Push(b);
        }
        else if(a == 1){
            tree.Search(b);
        }
        else if(a == 2){
            tree.DeleteElement(b);
        }
        else if(a == 3){
            tree.rankSearch(b);
        }
        else if(a == 4){
            tree.rankDelete(b);
        }
    }
    return 0;
}
```